1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

1.Data type of all columns in the "customers" table.

Field name	Type	Mode	Key
customer_id	STRING	NULLABLE	-
customer_unique_id	STRING	NULLABLE	-
customer_zip_code_prefix	INTEGER	NULLABLE	-
customer_city	STRING	NULLABLE	-
customer_state	STRING	NULLABLE	-

Insights and Recommendations: "The Customers" table consist of 4 string and 1 integer data types. The data types are useful in finding out what type of data are stored in the table.

2. Get the time range between which the orders were placed.

QUERY:

```
SELECT  MIN(order_purchase_timestamp) as first_order_date,
        MAX(order_purchase_timestamp) as last_order_date
FROM `target.orders` ;
```

Row	first_order_date	last_order_date
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Insights and Recommendations: The time range of the orders placed are between 2016-09-04 21:15:19 UTC and 2018-10-17 17:30:18 UTC. This is useful in finding out the time period when customers started placing their orders.

3. Count the Cities & States of customers who ordered during the given period.

QUERY:

```
SELECT FORMAT_DATE('%Y', o.order_purchase_timestamp) AS Year,
        COUNT(DISTINCT c.customer_city) AS city_count,
        COUNT(DISTINCT c.customer_state) AS state_count
FROM `target.orders` o INNER JOIN `target.customers` c ON o.customer_id = c.customer_id
GROUP BY Year
ORDER BY Year ASC;
```

Row	Year	city_count	state_count
1	2016	179	21
2	2017	3290	27
3	2018	3277	27

Insights and Recommendations: The count of different cities and states over the years is growing over the years. This is helpful in the resource allocations and inventory management so that there is neither wastage nor shortage of resources/inventory.

2. In-depth Exploration:

1. **Is there a growing trend in the no. of orders placed over the past years?**

QUERY:

```
SELECT  FORMAT_DATETIME("%Y", order_purchase_timestamp) AS year,
        COUNT(*) AS orders_placed
FROM `target.orders`
GROUP BY year
ORDER BY year ASC;
```

Row	year	orders_placed
1	2016	329
2	2017	45101
3	2018	54011

Insights and Recommendations: Yes, there is a growing trend in the number of orders placed over the years. This growing trend indicates the inventory and personnel should be increased to meet with the demands.

2. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

QUERY:

```
SELECT  FORMAT_DATETIME("%m", order_purchase_timestamp) AS month,
         COUNT(*) AS orders_placed
FROM `target.orders`
GROUP BY month
ORDER BY month ASC;
```

Row	month	orders_placed
1	01	8069
2	02	8508
3	03	9893
4	04	9343
5	05	10573
6	06	9412
7	07	10318
8	08	10843
9	09	4305
10	10	4959
11	11	7544
12	12	5674

Insights and Recommendations: Yes, there is a monthly seasonality in terms of the orders placed. August has the highest number of orders and September has the least number of orders. This

is very helpful in the inventory management so that the demand is met in the particular months and the promotions can be introduced to boost the sales on the months where the sales are low.

3.During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

QUERY:

```
WITH raw as (
SELECT  CAST(FORMAT_DATETIME("%H", order_purchase_timestamp) AS INT) AS hour,
        COUNT(*) AS orders_placed
FROM `target.orders`
GROUP BY hour
),

sq_time_of_day AS (
SELECT
    CASE WHEN hour BETWEEN 00 AND 06 THEN 'Dawn'
         WHEN hour BETWEEN 7 AND 12 THEN 'Mornings'
         WHEN hour BETWEEN 13 AND 18 THEN 'Afternoon'
         WHEN hour BETWEEN 19 AND 23 THEN 'Night'
     END as time_of_day,
     orders_placed
FROM raw
ORDER BY 2 DESC
)

SELECT time_of_day,
        SUM(orders_placed) AS total_orders_placed
FROM sq_time_of_day
GROUP BY time_of_day
ORDER BY total_orders_placed DESC
LIMIT 1;
```

Row	time_of_day	total_orders_placed
1	Afternoon	38135

Insights and Recommendations: Afternoon is the most preferred time for placing the orders for the Brazilian customers. The promotions can be introduced at the rest of time period so that the number of orders placed can be raised during the rest of the day.

3. Evolution of E-commerce orders in the Brazil region:

1. Get the month-on-month no. of orders placed in each state.

QUERY:

```
SELECT c.customer_state as state,
        FORMAT_DATE("%Y-%m", order_purchase_timestamp) as month,
        COUNT(*) as orders_placed
FROM `target.orders`  o INNER JOIN `target.customers` c ON o.customer_id = c.customer_id
```

```
GROUP BY state, month
ORDER BY month ASC, state ASC;
```

Row	state	month	orders_placed
1	RR	2016-09	1
2	RS	2016-09	1
3	SP	2016-09	2
4	AL	2016-10	2
5	BA	2016-10	4
6	CE	2016-10	8
7	DF	2016-10	6
8	ES	2016-10	4
9	GO	2016-10	9
10	MA	2016-10	4

Insights and Recommendations: The month-on-month orders placed for each state is given above. This data is useful in finding out the number of orders placed by the state over the months so that we can find out which state is doing well and which isn't over the months.

2. How are the customers distributed across all the states?

QUERY:

```
SELECT c.customer_state AS state,
       COUNT(DISTINCT customer_unique_id) AS no_cust
FROM  `target.customers` c
GROUP BY state
ORDER BY no_cust DESC
```

Row	state	no_cust
1	SP	40302
2	RJ	12384
3	MG	11259
4	RS	5277
5	PR	4882
6	SC	3534
7	BA	3277
8	DF	2075
9	ES	1964
10	GO	1952

Insights and Recommendations: The state 'SP' has the highest number of customers. Hence the resources/inventory should be distributed as per the demand. The new customer offers can be introduced in the rest of the states where the customer base is low to attract more customers.

4. Impact on Economy: Analyse the money movement by e-commerce by looking at order prices, freight and others.

1. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

QUERY:

```
WITH raw_data AS(
                SELECT EXTRACT(DATE FROM order_purchase_timestamp) AS order_date,
                ROUND(SUM(payment_value),2) AS total_cost

              FROM `target.orders` o INNER JOIN `target.payments` p ON o.order_id =  p.order_id
              GROUP BY order_date
              ORDER BY total_cost DESC
             ),
date_bin AS (
             SELECT order_date,
                 total_cost
             FROM raw_data
             WHERE order_date BETWEEN '2017-01-01' AND '2017-08-31' OR order_date BETWEEN
'2018-01-01' AND '2018-08-31'
             ORDER BY order_date ASC
          ),
extract_year as (
             SELECT EXTRACT(YEAR FROM order_date) AS year,
                       ROUND(SUM(total_cost),2) as curr_yr_cost
             FROM date_bin
             GROUP BY year
             ORDER BY year ASC
             ),
cal_nxt as (
            SELECT  year,
                 curr_yr_cost,
                 LEAD(curr_yr_cost,1) OVER(ORDER BY year) as nxt_yr_cost
            FROM extract_year
            ORDER BY year ASC
          )

SELECT
         ROUND(100*(nxt_yr_cost - curr_yr_cost)/curr_yr_cost,2) as perc_increase
FROM cal_nxt
WHERE nxt_yr_cost IS NOT NULL;
```

Row	perc_increase
1	136.98

Insights and Recommendations: There is 136.98% increase in the cost of orders from 2017 to 2018 which means that there is a clear demand and the supply should be increased to meet up with the demand.

2. Calculate the Total & Average value of order price for each state.

QUERY:

```
SELECT c.customer_state AS state,
        ROUND(SUM(oi.price),2) AS total_order_price,
        ROUND(SUM(oi.price)/COUNT(DISTINCT o.order_id),2) AS avg_order_price

FROM `target.order_items` oi INNER JOIN `target.orders` o ON oi.order_id = o.order_id  INNER JOIN
`target.customers` c ON c.customer_id = o.customer_id
GROUP BY state
ORDER BY total_order_price DESC;
```

Row	state	total_order_price	avg_order_price
1	SP	5202955.05	125.75
2	RJ	1824092.67	142.93
3	MG	1585308.03	137.33
4	RS	750304.02	138.13
5	PR	683083.76	136.67
6	SC	520553.34	144.12
7	BA	511349.99	152.28
8	DF	302603.94	142.4
9	GO	294591.95	146.78
10	ES	275037.31	135.82

Insights and Recommendations: The state 'SP' has the highest total order price among the states.

3.Calculate the Total & Average value of order freight for each state.

QUERY:

```
SELECT    c.customer_state AS state,
          ROUND(SUM(oi.freight_value),2) AS total_fv,
          ROUND(SUM(oi.freight_value)/COUNT(DISTINCT oi.order_id),2) AS avg_fv
FROM `target.order_items` oi INNER JOIN `target.orders` o ON oi.order_id = o.order_id INNER JOIN
`target.customers` c ON o.customer_id = c.customer_id
GROUP BY state
ORDER BY total_fv DESC;
```

Row	state	total_fv	avg_fv
1	SP	718723.07	17.37
2	RJ	305589.31	23.95
3	MG	270853.46	23.46
4	RS	135522.74	24.95
5	PR	117851.68	23.58
6	BA	100156.68	29.83
7	SC	89660.26	24.82
8	PE	59449.66	36.07
9	GO	53114.98	26.46
10	DF	50625.5	23.82

Insights and Recommendations: The state 'SP' has the highest freight order value when compared to the other states. The comparison can be done between the average freight values and the changes in freight services can be done to reduce the freight charges.

5. Analysis based on sales, freight and delivery time.

1. Find the no. of days taken to deliver each order from the order's purchase date as delivery time. Also, calculate the difference (in days) between the estimated & actual delivery date of an order. Do this in a single query.

QUERY:

```
SELECT order_id,
       DATE_DIFF(EXTRACT(DATE FROM order_delivered_customer_date), EXTRACT(DATE FROM order_purchase_timestamp ), DAY) AS time_to_deliver,
       DATE_DIFF(EXTRACT(DATE FROM order_delivered_customer_date), EXTRACT (DATE FROM order_estimated_delivery_date), DAY) AS diff_estimated_delivery
    FROM `target.orders`;
```

Row	order_id	time_to_deliver	diff_estimated_delivery
1	65d1e226dfaeb8cdc42f665422...	36	-17
2	2c45c33d2f9cb8ff8b1c86cc28...	31	-29
3	1950d777989f6a877539f53795...	30	12
4	bfbd0f9bdef84302105ad712db...	55	36
5	98974b076b01553d49ee64679...	44	-7
6	c4b41c36dd589e901f6879f25a...	36	-15
7	d2292ff2201e74c5db154d1b7a...	30	-21
8	95e01270fcbae986342340010...	31	-20
9	ed8c7b1b3eb256c70ce0c7423...	45	-6
10	5cc475c7c03290048eb2e742c...	69	18

Insights and Recommendations: The number of days it took to deliver and difference in estimated and actual delivery date is given.

2. Find out the top 5 states with the highest & lowest average freight value.

QUERY:

```
WITH freight_data AS(
        SELECT c.customer_state AS state,
            ROUND(SUM(oi.freight_value)/COUNT(DISTINCT oi.order_id),2) AS avg_freight_value
        FROM `target.order_items` oi INNER JOIN `target.orders` o ON oi.order_id = o.order_id
            INNER JOIN `target.customers` c ON c.customer_id = o.customer_id
        GROUP BY state
       ),

top_high AS (
        SELECT state,
            avg_freight_value,
            ROW_NUMBER() OVER(ORDER BY avg_freight_value DESC) AS rnk,
            'highest'AS category
        FROM freight_data
        ORDER BY avg_freight_value DESC
        LIMIT 5
      ),

top_low AS (
        SELECT state,
            avg_freight_value ,
            ROW_NUMBER() OVER(ORDER BY avg_freight_value ASC) AS rnk,
            'lowest' AS category
        FROM freight_data
        ORDER BY avg_freight_value ASC
        LIMIT 5
      )

SELECT state,
    avg_freight_value,
    rnk as rank,
    category
FROM top_high
UNION ALL
SELECT state,
    avg_freight_value ,
    rnk as rank,
    category
FROM top_low ;
```

Row	state	avg_freight_value	rank	category
1	SP	17.37	1	lowest
2	MG	23.46	2	lowest
3	PR	23.58	3	lowest
4	DF	23.82	4	lowest
5	RJ	23.95	5	lowest
6	RR	48.59	1	highest
7	PB	48.35	2	highest
8	RO	46.22	3	highest
9	AC	45.52	4	highest
10	PI	43.04	5	highest

Insights and Recommendations: The state 'SP' has the lowest average freight value and the state 'RR' has the highest average freight value.

3.Find out the top 5 states with the highest & lowest average delivery time.

QUERY:

```
WITH state_orders AS (
                SELECT o.order_id,
                    c.customer_state AS state,
                    EXTRACT(DATE FROM order_delivered_customer_date) AS delivered_date,
                    EXTRACT(DATE FROM order_purchase_timestamp) AS purchase_date,
                    DATE_DIFF(EXTRACT(DATE FROM order_delivered_customer_date), EXTRACT(DATE
FROM order_purchase_timestamp), DAY) AS delivery_time
                FROM `target.orders` o INNER JOIN `target.customers` c ON o.customer_id =
c.customer_id
              ),

high AS (
      SELECT state,
          ROUND(SUM(delivery_time)/COUNT(DISTINCT order_id),2) AS avg_delivery_time,
          'Highest' as category
      FROM state_orders
      GROUP BY state
      ORDER BY avg_delivery_time DESC
      LIMIT 5
    ),
low AS (
      SELECT state,
          ROUND(SUM(delivery_time)/COUNT(DISTINCT order_id),2) AS avg_delivery_time,
          'Lowest' AS category
      FROM state_orders
      GROUP BY state
      ORDER BY avg_delivery_time ASC
      LIMIT 5
    )

SELECT state,
    avg_delivery_time,
```

```
    category,
    ROW_NUMBER() OVER(ORDER BY avg_delivery_time DESC) AS rank
FROM high
UNION ALL
SELECT state,
    avg_delivery_time,
    category,
    ROW_NUMBER() OVER(ORDER BY avg_delivery_time ASC) AS rank
FROM low;
```

Row	state	avg_delivery_time	category	rank
1	AP	26.78	Highest	1
2	RR	26.15	Highest	2
3	AM	25.82	Highest	3
4	AL	23.55	Highest	4
5	PA	23.02	Highest	5
6	SP	8.44	Lowest	1
7	PR	11.65	Lowest	2
8	MG	11.66	Lowest	3
9	DF	12.54	Lowest	4
10	SC	14.54	Lowest	5

Insights and Recommendations: The state 'AP' has the highest average delivery time and the state 'SP' has the lowest average delivery time.

4. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.
You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

QUERY:

```
WITH state_orders AS (
SELECT c.customer_state AS state,
    o.order_id,
    DATE_DIFF(EXTRACT(DATE FROM order_delivered_customer_date), EXTRACT(DATE FROM
order_estimated_delivery_date), DAY) AS del_diff
FROM `target.orders` o INNER JOIN `target.customers` c ON o.customer_id = c.customer_id
WHERE order_delivered_customer_date IS NOT NULL
)

SELECT state,
    ROUND(AVG(del_diff),2) AS avg_del_diff
FROM state_orders
GROUP BY state
ORDER BY avg_del_diff ASC
LIMIT 5;
```

Row	state	avg_del_diff
1	AC	-20.72
2	RO	-20.1
3	AP	-19.69
4	AM	-19.57
5	RR	-17.29

Insights and Recommendations: The state 'AC' has the fastest order delivery among the states. The delivery carrier used in the state 'AC' can be used in other states to reduce the delivery time.

6.Analysis based on the payments:

1.Find the month-on-month no. of orders placed using different payment types.

QUERY:

```
SELECT
    FORMAT_TIMESTAMP("%Y-%m", order_purchase_timestamp) AS month,
    p.payment_type,
    COUNT(o.order_id) AS no_orders

FROM `target.orders` o INNER JOIN `target.payments` p ON o.order_id = p.order_id
GROUP BY month,p.payment_type
ORDER BY month,p.payment_type;
```

Row	month	payment_type	no_orders
1	2016-09	credit_card	3
2	2016-10	UPI	63
3	2016-10	credit_card	254
4	2016-10	debit_card	2
5	2016-10	voucher	23
6	2016-12	credit_card	1
7	2017-01	UPI	197
8	2017-01	credit_card	583
9	2017-01	debit_card	9
10	2017-01	voucher	61

Insights and Recommendations: The orders placed month-on-month for each payment type is given. This can be used in identifying the most preferred payment type by the customers over the months. Based on this the offers can be provided to encourage the other payments methods and infrastructure can be improved for the most preferred method.

2.Find the no. of orders placed on the basis of the payment instalments that have been paid.

QUERY:

```
SELECT  payment_installments,
      COUNT(DISTINCT order_id) AS no_of_orders
FROM `target.payments`
WHERE payment_installments > 0
GROUP BY payment_installments
ORDER BY payment_installments
```

Row	payment_installments	no_of_orders
1	1	49060
2	2	12389
3	3	10443
4	4	7088
5	5	5234
6	6	3916
7	7	1623
8	8	4253
9	9	644
10	10	5315

Insights and Recommendations: The number of orders placed on the basis of instalments are given. This is helpful in finding the preferred choice of instalment tenure by the customers.